

Intelligente Robotik — Praktikum

Localising a mecanum robot: Monte-Carlo localisation and ICP scan matching

Sheets L1—L4 (4 tasks)

Total 130 pts, pass from 50%. One sheet per task; hand in one protocol per person (max. 2 pages) — the code is shared, the explanation is not. Every number below has to be reproducible from a command you wrote down.

Name	
Date	
Group	
Seed	

How the numbers are produced The graded run is one number and takes about 36 s. One recording of the same drive turns every question after it into a three-second replay, so the sweep tables below are a loop over `mcl_report.py`, not four lab sessions.

```
./tools/run_lab.sh grade --task mcl_production --controller student/mcl_template.py --headless  
OHM_RECORD=/tmp/l1.jsonl ./tools/run_lab.sh grade --task mcl_production \  
    --controller tools/record_scans.py --headless # one drive, everything it saw  
./tools/mcl_report.py /tmp/l1.jsonl --particles 250 --stride 1 --sigma-z 0.5 --timing
```

1 L1 — Where am I? Monte-Carlo localization against the map

30 PTS hall: production task: mcl_production

Task

The grader drives; you estimate. This time the improvement is measured against your **odometry**, not against GPS — the wheels are the sensor that is already fused into the pose you start with, and beating them is the whole claim of the exercise. One scan every twentieth of a second, 360 beams, and a map you did not build: draw N samples from the belief you were given, move them by the odometry, weight each by how well its predicted beams fit the walls, and resample when the weights stop being spread out. Report the mean and the 1σ on `<robot>/kf/pose` — the σ is graded (NEES), so a cloud that is wide enough to hide in is not a passing answer.

What is graded

quantity	target (from the task file)
accuracy (RMSE over the graded window)	≤ 50 mm
improvement over raw odometry	$\geq 5\times$
worst single error	≤ 250 mm
rate of kf/pose	≥ 5 Hz
wall contacts	≤ 0
NEES (σ honest)	[0.05–5]

Protocol — one table per item. Four blank rows each; add rows if you need them, and write the command that produced a number under the table.

C1.1 *Particles are moved by the odometry's own delta (rot1, translate, rot2), not teleported to the odometry pose*

question / item	method	measured	target	explanation of the deviation

C2.1 *Weight of a particle from the map: predicted beam end points compared against the distance field*

question / item	method	measured	target	explanation of the deviation

C3.1 $N_{eff} = 1/\Sigma w^2$ as the trigger for resampling, and the resampling does not happen on every scan

question / item	method	measured	target	explanation of the deviation

C4.1 $kf/pose$ with sx/sy from the spread of the cloud, not a constant

question / item	method	measured	target	explanation of the deviation

Pointers

- Beam angles: `angle_min + i*angle_increment + theta` of the particle — a beam that keeps the robot's heading instead of the particle's weights every hypothesis as if it were the odometry.
- Use the beams that are left after you drop `inf`; with $\sigma_z = 0.15$ m and ~110 usable beams a particle inside a wall and one on the correct pose differ by dozens of orders of magnitude, which is why the weights are summed in log form.
- The scatter you add at the motion step is not decoration: with zero motion noise the cloud can only shrink, and the filter ends up certain about the odometry's own drift.
- σ from the cloud is the weighted standard deviation around the circular mean of the heading — the arithmetic mean of $+170^\circ$ and -170° points the wrong way.

Checkpoint

checkpoint	demo shown	assistant initials	date

AI tools used (language, debugging, API questions only, never the reference solution)_____

2 L2 — Parked somewhere else: localisation from a 2 m prior

35 PTS hall: production task: mcl_wide

Task

Same hall, same drive, and the robot was not where the odometry claims: the prior around the start pose has a 1σ of two metres instead of half a metre, so the true pose is in the cloud but surrounded by a thousand wrong neighbours. What has to survive is the weighting — the first scans have to throw away the neighbours. `max_error` is the number that grades the beginning of the run rather than its average: a filter that takes eight seconds to find the robot has a good RMSE and a useless first eight seconds.

What is graded

quantity	target (from the task file)
accuracy (RMSE over the graded window)	≤ 60 mm
improvement over raw odometry	$\geq 3\times$
worst single error	≤ 350 mm
rate of kf/pose	≥ 5 Hz
wall contacts	≤ 0
NEES (σ honest)	[0.05–8]

Protocol — one table per item. Four blank rows each; add rows if you need them, and write the command that produced a number under the table.

C1.2 *The initial cloud is drawn from the prior of the task, not placed on the odometry pose with zero spread*

question / item	method	measured	target	explanation of the deviation

C2.2 *The first scans reduce N_{eff} and then the resampling concentrates the cloud (watch it in RViz/the window)*

question / item	method	measured	target	explanation of the deviation

C3.2 *No use of `/<robot>/truth`, and no GPS: this task is decided by the LIDAR and the map*

question / item	method	measured	target	explanation of the deviation

C4.2 Report of how many scans the first 25 cm took — from the counters on kf/info, not from a stopwatch

question / item	method	measured	target	explanation of the deviation

Pointers

- A prior that is too narrow is not optimism, it is a different filter: if no particle is within σ_z of the true pose, no amount of weighting will find it.
- Look at N_{eff} while it converges. Falling and then rising again is the shape of a filter that found the robot; sitting at 1 is a filter that found one lucky particle instead.
- If you are tempted to add random particles to make this pass: measure what that costs in L1 first (see docs/mcl.md) — injection is a recovery mechanism, not a performance knob.

Checkpoint

checkpoint	demo shown	assistant initials	date

AI tools used (language, debugging, API questions only, never the reference solution)_____

3 L3 — Same accuracy, a quarter of the particles: where the budget belongs

35 PTS hall: production task: mcl_budget

Task

The task file now says 250 particles and **every** beam instead of one in three — a quarter of the particles for nearly the same accuracy, and $3\times$ the beams per particle. Both settings cost the same per scan (particles $\times$ beams is the product that sets the load), so this is the same computation spent differently, and the accuracy you get for it is not the same: 360 beams constrain a pose far more sharply than 1200 particles cover a hall. The thresholds below are L1's, loosened by as much as a quarter of a cloud measurably costs — that difference is itself a result worth a line in your protocol. Run `tools/mcl_report.py -timing` on your own filter and hand in the table of what each knob costs.

What is graded

quantity	target (from the task file)
accuracy (RMSE over the graded window)	≤ 70 mm
improvement over raw odometry	$\geq 3\times$
worst single error	≤ 300 mm
rate of kf/pose	≥ 5 Hz
wall contacts	≤ 0
NEES (σ honest)	[0.05–12]

Protocol — one table per item. Four blank rows each; add rows if you need them, and write the command that produced a number under the table.

C1.3 *One update: all particles and all beams in vectorised NumPy, no Python loop over particles*

question / item	method	measured	target	explanation of the deviation

C2.3 *The product particles $\times$ beams as the cost of one update, measured rather than estimated*

question / item	method	measured	target	explanation of the deviation

C3.3 *What broke first when the particles were taken away: N_{eff} , the RMSE, or the maximum error*

quantity	value	how it was measured	what could make it wrong

C4.3 *A short answer to why 360 beams are not 360 independent measurements of a hall*

question / item	method	measured	target	explanation of the deviation

Pointers

- The work of one update is (particles $\times$ beams) gathers in the distance field. Halving one factor and tripling the other keeps the work the same and changes what the filter is good at.
- Fewer particles means a coarser sampling of the pose space: σ_z controls how forgiving the likelihood is about that coarseness, and it is the knob that lets 250 particles find a 1 cm peak.
- Report N_{eff} for both settings. If it did not go down with a quarter of the cloud, the resampling is doing something you did not intend.

Checkpoint

checkpoint	demo shown	assistant initials	date

AI tools used (language, debugging, API questions only, never the reference solution)_____

4 L4 — The window is dirty: make the model match the sensor

30 PTS hall: production task: mcl_dirty

Task

The same hall, the same drive and the same filter as L1, with one change you will not see in the topic: the LIDAR's σ is 0.25 m instead of 0.015 m — a reflective, partially covered window, which on a real robot is a Tuesday. The parameters that won L1 were tuned against a clean sensor and they will not survive this one. Not because the filter is broken, but because σ_z is not a knob on the filter, it is a claim about the sensor: weight a 250 mm measurement as if it were a 150 mm measurement and the cloud collapses onto whatever the first dirty beams liked, and the reported σ becomes a lie even while the position happens to stay reasonable. Find parameters that work with this sensor and write down which one you changed and why that one.

What is graded

quantity	target (from the task file)
accuracy (RMSE over the graded window)	≤ 70 mm
improvement over raw odometry	$\geq 2.5\times$
worst single error	≤ 200 mm
rate of kf/pose	≥ 5 Hz
wall contacts	≤ 0
NEES (σ honest)	[0.3–5]

Protocol — one table per item. Four blank rows each; add rows if you need them, and write the command that produced a number under the table.

C1.4 *RMSE and improvement over the raw odometry, with the parameters that achieved them*

run / settings	RMSE [mm]	raw odom [mm]	improvement [-]	NEES [-]	verdict

C2.4 *NEES: does your σ describe your error now, and what happened to it when you changed σ_z*

configuration	reported σ [mm]	actual error [mm]	NEES [-]	should be [-]

C3.4 *A σ_z sweep — at least four values, one number each — as a table in the protocol*

parameter	value tried	RMSE [mm]	improvement [-]	NEES [-]	comment

C4.4 Why $\sigma_z \approx 0.5$ m works better here than $\sigma_z = 0.25$ m, which is the σ the sensor actually has

question / item	method	measured	target	explanation of the deviation

C5.4 What z_rand costs you when the sensor is this noisy

quantity	value	how it was measured	what could make it wrong

Pointers

- The measurement noise is in the task's `sim` block, and the grader checks that the run really used it: `lidar.sigma` is 0.25 m. Read it off the config rather than assuming 0.015 m — a filter that asks the bus what its sensor can do is a filter that survives a sensor change.
- σ_z enters the weight of every beam, so it is the parameter this task is about. Everything else can stay where L1 left it.
- Measured on this recording, for comparison once you have your own numbers: σ_z 0.15 $\rightarrow$ 96 mm and NEES $\sim$ 15; σ_z 0.30 $\rightarrow$ 62 mm; σ_z 0.50 $\rightarrow$ 49 mm and NEES 2.2; σ_z 0.80 $\rightarrow$ 62 mm. There is a minimum in there, and it is near $2 \cdot \sigma_{\text{sensor}}$ rather than at σ_{sensor} .
- `tools/mcl_report.py` replays a recording in seconds, so a sweep of four σ_z values is a loop, not four lab sessions.

Checkpoint

checkpoint	demo shown	assistant initials	date

AI tools used (language, debugging, API questions only, never the reference solution) _____